

LAPORAN PRAKTIKUM 5
STRUKTUR DATA
IMPLEMENTASI *ABSTRACT DATA TYPE (ADT)*
SINGLE LINKEDLIST* MENGGUNAKAN *JAVA



Disusun Oleh:

Naysha Aprilia Rizki

2511531014

Dosen Pengampu:

Wahyudi. Dr. S.T.M.T

Asisten Praktikum:

Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum yang berjudul “Laporan Praktikum 5 Struktur Data: Implementasi Abstract Data Type (ADT) Single LinkedList Menggunakan Bahasa Java” ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum pada mata kuliah Struktur Data. Adapun tujuan dari praktikum ini adalah untuk memahami konsep Abstract Data Type (ADT), khususnya pada struktur data **Single Linked List**, serta mengimplementasikannya menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga bertujuan agar mahasiswa mampu memahami cara kerja struktur data berbasis pointer, seperti proses penambahan (insert), penghapusan (delete), dan pencarian (search) data dalam suatu linked list.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca mengenai struktur data, khususnya penerapan **Single Linked List** dalam pemrograman Java.

Padang, 6 Mei 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Salah satu struktur data yang sering digunakan adalah **Single Linked List**. Single Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node berisi data dan sebuah pointer (referensi) yang menunjuk ke node berikutnya.

Berbeda dengan array yang memiliki ukuran tetap dan penyimpanan berurutan dalam memori, Single Linked List bersifat dinamis sehingga ukuran data dapat bertambah atau berkurang sesuai kebutuhan. Hal ini membuat Single Linked List lebih fleksibel dalam pengelolaan data, terutama ketika sering dilakukan operasi penambahan dan penghapusan data.

Dalam pemrograman, Single Linked List banyak digunakan untuk memanipulasi data secara dinamis, seperti proses penambahan (insert), penghapusan (delete), dan pencarian (search) data. Implementasi struktur data ini dalam bahasa pemrograman Java dilakukan dengan menggunakan konsep objek dan referensi, sehingga mahasiswa dapat memahami cara kerja pointer secara lebih nyata. Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep Single Linked List serta mampu mengimplementasikannya dalam bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar struktur data Single Linked List.
2. Mengimplementasikan Single Linked List dalam bahasa pemrograman Java.
3. Memahami operasi dasar pada Single Linked List seperti penambahan, penghapusan, dan pencarian data.
4. Melatih kemampuan pemrograman dalam mengelola data secara dinamis menggunakan struktur data.

1.3 Manfaat

1. Menambah pemahaman tentang struktur data Single Linked List.
2. Menjadi dasar pembelajaran untuk struktur data lain yang lebih kompleks.
3. Membantu memahami pengelolaan data secara dinamis dalam program.

BAB II

PEMBAHASAN

Dalam pemrograman, struktur data merupakan cara untuk menyimpan dan mengorganisasi data agar dapat digunakan secara efisien. Salah satu konsep penting dalam struktur data adalah **Abstract Data Type (ADT)**. ADT adalah suatu tipe data abstrak yang mendefinisikan sekumpulan nilai dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana implementasinya secara detail.

Konsep utama dari ADT adalah pemisahan antara interface (apa yang dapat dilakukan) dan implementasi (bagaimana cara melakukannya). Dengan menggunakan ADT, programmer dapat lebih fokus pada fungsi dan kegunaan suatu tipe data tanpa harus memahami detail internalnya. Hal ini membuat program menjadi lebih modular, mudah dipelihara, dan mudah dikembangkan.

Dalam bahasa pemrograman Java, ADT biasanya direpresentasikan menggunakan class. Sebuah class berisi atribut (data) dan metode (fungsi) yang mendefinisikan perilaku dari objek tersebut. Atribut digunakan untuk menyimpan data, sedangkan metode digunakan untuk melakukan operasi terhadap data tersebut.

Pada praktikum ini, ADT yang diimplementasikan adalah ADT Single Linked List. Single Linked List merupakan struktur data linear yang terdiri dari sekumpulan node, di mana setiap node memiliki dua bagian, yaitu data dan pointer (referensi) yang menunjuk ke node berikutnya. Struktur ini bersifat dinamis karena ukuran data dapat berubah sesuai kebutuhan, berbeda dengan array yang memiliki ukuran tetap.

Single Linked List digunakan untuk mengelola data secara berurutan, di mana setiap elemen saling terhubung melalui pointer. Akses data dimulai dari node pertama yang disebut head, kemudian dilanjutkan ke node berikutnya hingga mencapai node terakhir yang memiliki pointer bernilai null.

Selain itu, ADT Single Linked List juga dilengkapi dengan beberapa operasi, antara lain:

1. Penambahan data di depan (**insert at front**), yaitu menambahkan node baru di awal list.
2. Penambahan data di belakang (**insert at end**), yaitu menambahkan node baru di akhir list.
3. Penambahan data pada posisi tertentu (**insert at position**), yaitu menyisipkan node pada posisi tertentu dalam list.

4. Penghapusan data (**delete**), yaitu menghapus node baik di awal, akhir, maupun pada posisi tertentu.
5. Pencarian data (**search**), yaitu mencari apakah suatu data terdapat dalam list.
6. Penelusuran data (**traversal**), yaitu menampilkan seluruh isi linked list secara berurutan.

Dalam implementasinya, Single Linked List menggunakan konsep referensi (pointer) antar node. Setiap node akan menunjuk ke node berikutnya sehingga membentuk suatu rangkaian. Keunggulan dari struktur ini adalah fleksibilitas dalam penambahan dan penghapusan data tanpa perlu menggeser elemen seperti pada array. Namun, kelemahannya adalah akses data tidak dapat dilakukan secara langsung (random access), melainkan harus ditelusuri satu per satu dari awal.

Dengan memahami dan mengimplementasikan ADT Single Linked List dalam bahasa Java, mahasiswa dapat mengembangkan kemampuan dalam mengelola data secara dinamis serta meningkatkan pemahaman terhadap konsep dasar struktur data dalam pemrograman.

1. Langkah Kerja

1.1 NodeSLL_2511531014

- Kode Program

```
1 package pekan5_2511531014;
2
3 public class NodeSLL_2511531014 {
4     //node bagian data
5     int data_1014;
6     //pointer ke node berikutnya
7     NodeSLL_2511531014 next_1014;
8     //Konstruktor menganalisis node dengan data
9     NodeSLL_2511531014(int data) {
10         this.data_1014 = data;
11         this.next_1014 = null;
12     }
13 }
```

- Pembahasan

Program ini dibuat dengan mendefinisikan class NodeSLL_2511531014 yang digunakan sebagai representasi node pada struktur data *Singly Linked List* (SLL). Class ini berfungsi sebagai elemen dasar yang menyusun linked list.

1. Class ini memiliki atribut int data_1014 yang digunakan untuk menyimpan nilai data pada node, serta atribut NodeSLL_2511531014

next_1014 yang berfungsi sebagai pointer atau referensi ke node berikutnya dalam linked list.

```
1 package pekan5_2511531014;
2
3 public class NodeSLL_2511531014 {
4     //node bagian data
5     int data_1014;
6     //pointer ke node berikutnya
7     NodeSLL_2511531014 next_1014;
```

2. Konstruktor NodeSLL_2511531014(int data) digunakan untuk menginisialisasi node dengan nilai tertentu. Pada konstruktor ini, atribut data_1014 diisi dengan nilai yang diberikan, sedangkan next_1014 diatur bernilai null yang menandakan bahwa node tersebut belum terhubung dengan node lain. Penggunaan keyword this pada konstruktor digunakan untuk membedakan antara atribut class dan parameter yang memiliki nama yang sama, sehingga nilai dapat disimpan dengan benar ke dalam atribut.

```
//Konstruktor menganalisis node dengan data
NodeSLL_2511531014(int data) {
    this.data_1014 = data;
    this.next_1014 = null;
}
```

3. Secara keseluruhan, class ini digunakan sebagai komponen utama dalam pembentukan struktur data Singly Linked List, di mana setiap node menyimpan data dan referensi ke node berikutnya, sehingga memungkinkan penyimpanan data secara dinamis dan terhubung

1.2 TambahSLL_2511531014

- Kode Program

```
1 package pekan5_2511531014;
2
3 public class TambahSLL_2511531014 {
4
5     public static NodeSLL_2511531014 insertAtFront(NodeSLL_2511531014 head, int value) {
6         NodeSLL_2511531014 newNode_1014 = new NodeSLL_2511531014(value);
7         newNode_1014.next_1014 = head;
8         return newNode_1014;
9     }
10
11     // fungsi menambahkan node di akhir SLL
12     public static NodeSLL_2511531014 insertAtEnd(NodeSLL_2511531014 head, int value) {
13         NodeSLL_2511531014 newNode_1014 = new NodeSLL_2511531014(value);
14
15         if (head == null) {
16             return newNode_1014;
17         }
18         NodeSLL_2511531014 last = head;
19
20         while (last.next_1014 != null) {
21             last = last.next_1014;
22         }
23         last.next_1014 = newNode_1014;
24         return head;
25     }
26
27     static NodeSLL_2511531014 getNode(int data_1014) {
28         return new NodeSLL_2511531014(data_1014);
29     }
30
31     static NodeSLL_2511531014 insertPos(NodeSLL_2511531014 headNode, int position, int value) {
32         NodeSLL_2511531014 head = headNode;
33
34         if (position < 1) {
35             System.out.print("Invalid position");
36             return head;
37         }
38     }
39 }
40
```

```

41● if (position == 1) {
42    NodeSLL_2511531014 newnode_1014 = new NodeSLL_2511531014(value);
43    newnode_1014.next_1014 = head;
44    return newnode_1014;
45● } else {
46● while (position-- != 0 && headNode != null) {
47● if (position == 1) {
48    NodeSLL_2511531014 newnode_1014 = getNode(value);
49    newnode_1014.next_1014 = headNode.next_1014;
50    headNode.next_1014 = newnode_1014;
51    break;
52    }
53    headNode = headNode.next_1014;
54    }
55    if (position != 1) {
56        System.out.print("Posisi di luar jangkauan");
57    }
58    return head;
59    }
60    }
61    }
62    }
63
64● public static void printList(NodeSLL_2511531014 head) {
65    NodeSLL_2511531014 curr = head;
66
67● while (curr != null) {
68    System.out.print(curr.data_1014);
69● if (curr.next_1014 != null) {
70    System.out.print("-->");
71    }
72    curr = curr.next_1014;
73    }
74    System.out.println();
75    }
76
77● public static void main(String[] args) {
78    // buat linked list 2->3->5->6
79    NodeSLL_2511531014 head = new NodeSLL_2511531014(2);
80    head.next_1014 = new NodeSLL_2511531014(3);
81    head.next_1014.next_1014 = new NodeSLL_2511531014(5);
82    head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(6);
83
84    // cetak list asli
85    System.out.print("Senarai berantai awal:");
86    printList(head);
87
88    // tambah node baru di depan
89    System.out.print("tambah 1 simpul di depan: ");
90    int data = 1;
91    head = insertAtFront(head, data);
92    printList(head);
93
94    // tambahkan node baru di belakang
95    System.out.print("tambah 1 simpul di belakang: ");
96    int data2 = 7;
97    head = insertAtEnd(head, data2);
98    printList(head);
99
100    System.out.print("tambah 1 simpul ke data 4: ");
101    int data3 = 4;
102    int pos = 4;
103    head = insertPos(head, pos, data3);
104    printList(head);
105    }
106 }

```

- Pembahasan

Program ini dibuat dengan mendefinisikan class `TambahSLL_2511531014` yang berfungsi untuk melakukan berbagai operasi penambahan node pada struktur data *Singly Linked List* (SLL). Program ini memanfaatkan class `NodeSLL_2511531014` sebagai representasi node yang menyimpan data dan pointer ke node berikutnya.

1. Method `insertAtFront(NodeSLL_2511531014 head, int value)` digunakan untuk menambahkan node di bagian depan linked list. Pada method ini dibuat node baru, kemudian pointer `next_1014` diarahkan ke head lama, dan node baru tersebut dikembalikan sebagai head yang baru.

```

- public static NodeSLL_2511531014 insertAtFront(NodeSLL_2511531014 head, int value) {
    NodeSLL_2511531014 newnode_1014 = new NodeSLL_2511531014(value);
    newnode_1014.next_1014 = head;
    return newnode_1014;
}

```

2. Method `insertAtEnd(NodeSLL_2511531014 head, int value)` digunakan untuk menambahkan node di bagian akhir linked list. Jika

head bernilai null, maka node baru langsung menjadi head. Jika tidak, dilakukan penelusuran hingga node terakhir menggunakan perulangan while, kemudian node baru ditambahkan di bagian akhir.

```
// fungsi menambahkan node di akhir SLL
public static NodeSLL_2511531014 insertAtEnd(NodeSLL_2511531014 head, int value) {
    NodeSLL_2511531014 newNode_1014 = new NodeSLL_2511531014(value);

    if (head == null) {
        return newNode_1014;
    }

    NodeSLL_2511531014 last = head;

    while (last.next_1014 != null) {
        last = last.next_1014;
    }

    last.next_1014 = newNode_1014;
    return head;
}
```

3. Method `GetNode(int data_1014)` digunakan sebagai fungsi pembantu untuk membuat node baru dengan nilai tertentu, sehingga mempermudah proses pembuatan node pada method lain.

```
static NodeSLL_2511531014 GetNode(int data_1014) {
    return new NodeSLL_2511531014(data_1014);
}
```

4. Method `insertPos(NodeSLL_2511531014 headNode, int position, int value)` digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Method ini melakukan pengecekan posisi, jika posisi pertama maka node ditambahkan di depan, sedangkan untuk posisi lainnya dilakukan penelusuran hingga posisi yang diinginkan, kemudian node baru disisipkan dengan mengatur pointer antar node.

```
static NodeSLL_2511531014 insertPos(NodeSLL_2511531014 headNode, int position, int value) {
    NodeSLL_2511531014 head = headNode;

    if (position < 1) {
        System.out.print("Invalid position");
        return head;
    }

    if (position == 1) {
        NodeSLL_2511531014 newNode_1014 = new NodeSLL_2511531014(value);
        newNode_1014.next_1014 = head;
        return newNode_1014;
    } else {
        while (position-- != 0 && headNode != null) {
            if (position == 1) {
                NodeSLL_2511531014 newNode_1014 = GetNode(value);
                newNode_1014.next_1014 = headNode.next_1014;
                headNode.next_1014 = newNode_1014;
                break;
            }
            headNode = headNode.next_1014;
        }

        if (position != 1) {
            System.out.print("Posisi di luar jangkauan");
        }

        return head;
    }
}
```


5. Method `printList(NodeSLL_2511531014 head)` digunakan untuk menampilkan seluruh isi linked list. Proses dilakukan dengan menelusuri node satu per satu menggunakan perulangan `while` hingga mencapai node terakhir, kemudian setiap data ditampilkan dengan tanda penghubung.

```
public static void printList(NodeSLL_2511531014 head) {
    NodeSLL_2511531014 curr = head;

    while (curr != null) {
        System.out.print(curr.data_1014);
        if (curr.next_1014 != null) {
            System.out.print("-->");
        }
        curr = curr.next_1014;
    }
    System.out.println();
}
```

6. Pada method `main`, dibuat linked list awal secara manual dengan beberapa node yang saling terhubung. Selanjutnya dilakukan pengujian terhadap setiap method, yaitu menambahkan node di depan, di belakang, dan pada posisi tertentu, kemudian hasilnya ditampilkan untuk memastikan bahwa operasi berjalan dengan benar.

```
public static void main(String[] args) {
    // buat linked list 2->3->5->6
    NodeSLL_2511531014 head = new NodeSLL_2511531014(2);
    head.next_1014 = new NodeSLL_2511531014(3);
    head.next_1014.next_1014 = new NodeSLL_2511531014(5);
    head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(6);

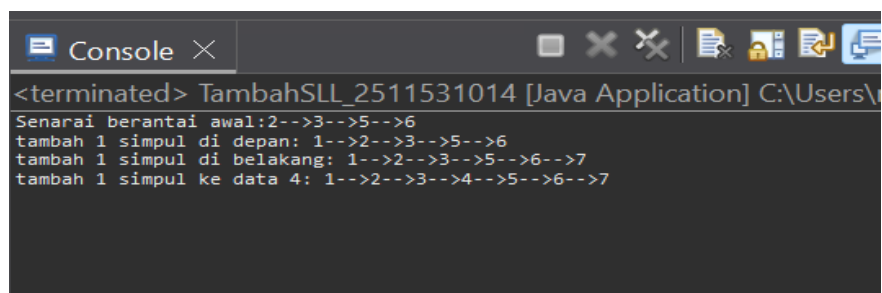
    // cetak list asli
    System.out.print("Senarai berantai awal:");
    printList(head);

    // tambah node baru di depan
    System.out.print("tambah 1 simpul di depan: ");
    int data = 1;
    head = insertAtFront(head, data);
    printList(head);

    // tambahkan node baru di belakang
    System.out.print("tambah 1 simpul di belakang: ");
    int data2 = 7;
    head = insertAtEnd(head, data2);
    printList(head);

    System.out.print("tambah 1 simpul ke data 4: ");
    int data3 = 4;
    int pos = 4;
    head = insertPos(head, pos, data3);
    printList(head);
}
```

7. Secara keseluruhan, program ini mengimplementasikan operasi penambahan pada struktur data Singly Linked List, Hasil Output sebagai berikut



```
<terminated> TambahSLL_2511531014 [Java Application] C:\Users\
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

1.3 HapusSLL_2511531014

- Kode Program

```
1 package pekan5_2511531014;
2
3 public class HapusSLL_2511531014 {
4
5     // fungsi untuk menghapus head
6     public static NodeSLL_2511531014 deleteHead(NodeSLL_2511531014 head) {
7         // jika SLL kosong
8         if (head == null)
9             return null;
10
11         // pindahkan head ke node berikutnya
12         head = head.next_1014;
13
14         // return head baru
15         return head;
16     }
17
18     // fungsi menghapus node terakhir SLL
19     public static NodeSLL_2511531014 removeLastNode(NodeSLL_2511531014 head) {
20
21         // jika list kosong
22         if (head == null) {
23             return null;
24         }
25
26         // jika hanya 1 node
27         if (head.next_1014 == null) {
28             return null;
29         }
30
31         // cari node ke-2 dari belakang
32         NodeSLL_2511531014 secondLast = head;
33
34         while (secondLast.next_1014.next_1014 != null) {
35             secondLast = secondLast.next_1014;
36         }
37
38         // hapus node terakhir
39         secondLast.next_1014 = null;
40
41         return head;
42     }
43
44     // fungsi menghapus node di posisi tertentu
45     public static NodeSLL_2511531014 deleteNode(NodeSLL_2511531014 head, int position) {
46         NodeSLL_2511531014 temp = head;
47         NodeSLL_2511531014 prev = null;
48
49         // jika linked list kosong
50         if (temp == null)
51             return head;
52
53         // kasus 1: hapus head
54         if (position == 1) {
55             head = temp.next_1014;
56             return head;
57         }
58
59         // keluarkan ke node yang mau dihapus
60         for (int i = 1; temp != null && i < position; i++) {
61             prev = temp;
62             temp = temp.next_1014;
63         }
64
65         // jika node ditemukan
66         if (temp != null) {
67             prev.next_1014 = temp.next_1014;
68         } else {
69             System.out.println("Data tidak ada");
70         }
71
72         return head;
73     }
74
75     // fungsi mencetak SLL
76     public static void printList(NodeSLL_2511531014 head) {
77         NodeSLL_2511531014 curr = head;
78
79         // handle kalau list kosong
80         if (curr == null) {
81             System.out.println("List kosong");
82             return;
83         }
84
85         while (curr != null) {
86             System.out.print(curr.data_1014);
87             if (curr.next_1014 != null) {
88                 System.out.print("-->");
89             }
90             curr = curr.next_1014;
91         }
92         System.out.println();
93     }
94
95     // kelas main
96     public static void main(String[] args) {
97         // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
98         NodeSLL_2511531014 head = new NodeSLL_2511531014(1);
99         head.next_1014 = new NodeSLL_2511531014(2);
100         head.next_1014.next_1014 = new NodeSLL_2511531014(3);
101         head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(4);
102         head.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(5);
103         head.next_1014.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(6);
104
105         // cetak list awal
106         System.out.println("List awal: ");
107         printList(head);
108
109         // hapus head
110         head = deleteHead(head);
111         System.out.println("List setelah head dihapus: ");
112         printList(head);
113
114         // hapus node terakhir
115         head = removeLastNode(head);
116         System.out.println("List setelah simpul terakhir dihapus: ");
117         printList(head);
118
119         // hapus node di posisi 2
120         int position = 2;
121         head = deleteNode(head, position);
122         System.out.println("List setelah posisi 2 dihapus: ");
123         printList(head);
124     }
125 }
```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan class HapusSLL_2511531014 yang berfungsi untuk melakukan operasi penghapusan node pada struktur data *Singly Linked List* (SLL). Program ini memanfaatkan class NodeSLL_2511531014 sebagai node yang menyimpan data dan pointer ke node berikutnya.

1. Method deleteHead(NodeSLL_2511531014 head) digunakan untuk menghapus node pertama (head) pada linked list. Jika list kosong maka method mengembalikan null, sedangkan jika tidak kosong maka head dipindahkan ke node berikutnya sehingga node pertama terhapus.

```
// fungsi untuk menghapus head
public static NodeSLL_2511531014 deleteHead(NodeSLL_2511531014 head) {
    // jika SLL kosong
    if (head == null)
        return null;

    // pindahkan head ke node berikutnya
    head = head.next_1014;

    // return head baru
    return head;
}
```

2. Method removeLastNode(NodeSLL_2511531014 head) digunakan untuk menghapus node terakhir pada linked list. Jika list kosong atau hanya memiliki satu node, maka akan menghasilkan list kosong. Jika lebih dari satu node, dilakukan penelusuran hingga node kedua dari belakang, kemudian pointer node tersebut diubah menjadi null untuk menghapus node terakhir.

```
// fungsi menghapus node terakhir SLL
public static NodeSLL_2511531014 removeLastNode(NodeSLL_2511531014 head) {

    // jika list kosong
    if (head == null) {
        return null;
    }

    // jika hanya 1 node
    if (head.next_1014 == null) {
        return null;
    }

    // cari node ke-2 dari belakang
    NodeSLL_2511531014 secondLast = head;

    while (secondLast.next_1014.next_1014 != null) {
        secondLast = secondLast.next_1014;
    }

    // hapus node terakhir
    secondLast.next_1014 = null;

    return head;
}
```

3. Method deleteNode(NodeSLL_2511531014 head, int position) digunakan untuk menghapus node pada posisi tertentu. Jika posisi

adalah 1, maka node pertama dihapus. Untuk posisi lainnya, dilakukan penelusuran menggunakan perulangan hingga mencapai node yang diinginkan, kemudian pointer diatur ulang sehingga node tersebut terlepas dari linked list.

```
// fungsi menghapus node di posisi tertentu
public static NodeSLL_2511531014 deleteNode(NodeSLL_2511531014 head, int position) {
    NodeSLL_2511531014 temp = head;
    NodeSLL_2511531014 prev = null;

    // jika linked list kosong
    if (temp == null)
        return head;

    // kasus 1: harus head
    if (position == 1) {
        head = temp.next_1014;
        return head;
    }

    // telusuri ke node yang mau dihapus
    for (int i = 1; temp != null && i < position; i++) {
        prev = temp;
        temp = temp.next_1014;
    }

    // jika node ditemukan
    if (temp != null) {
        prev.next_1014 = temp.next_1014;
    } else {
        System.out.println("Data tidak ada");
    }

    return head;
}
```

4. Method `printList(NodeSLL_2511531014 head)` digunakan untuk menampilkan seluruh isi linked list. Method ini juga menangani kondisi jika list kosong, kemudian menampilkan setiap data node secara berurutan dengan tanda penghubung antar node.

```
// fungsi mencetak SLL
public static void printList(NodeSLL_2511531014 head) {
    NodeSLL_2511531014 curr = head;

    // handle kalau list kosong
    if (curr == null) {
        System.out.println("List kosong");
        return;
    }

    while (curr != null) {
        System.out.print(curr.data_1014);
        if (curr.next_1014 != null) {
            System.out.print("-->");
        }
        curr = curr.next_1014;
    }
    System.out.println();
}
```

5. Pada method main, dibuat linked list secara manual dengan beberapa node yang saling terhubung. Selanjutnya dilakukan pengujian terhadap setiap operasi penghapusan, yaitu menghapus node pertama, node terakhir, dan node pada posisi tertentu, kemudian hasilnya ditampilkan untuk memastikan bahwa operasi berjalan dengan benar.

```

// kelas main
public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511531014 head = new NodeSLL_2511531014(1);
    head.next_1014 = new NodeSLL_2511531014(2);
    head.next_1014.next_1014 = new NodeSLL_2511531014(3);
    head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(4);
    head.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(5);
    head.next_1014.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(6);

    // cetak list awal
    System.out.println("List awal: ");
    printList(head);

    // hapus head
    head = deleteHead(head);
    System.out.println("List setelah head dihapus: ");
    printList(head);

    // hapus node terakhir
    head = removeLastNode(head);
    System.out.println("List setelah simpul terakhir dihapus: ");
    printList(head);

    // hapus node di posisi 2
    int position = 2;
    head = deleteNode(head, position);
    System.out.println("List setelah posisi 2 dihapus: ");
    printList(head);
}
}

```

6. Secara keseluruhan, program ini mengimplementasikan berbagai operasi penghapusan pada struktur data Singly Linked List, Hasil Output sebagai berikut

```

<terminated> HapusSLL_2511531014 [Java Application] C:\Users\naysh\
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5

```

1.4 PencarianSLL_2511531014

- Kode Program

```

1 package pekan5_2511531014;
2
3 public class PencarianSLL_2511531014 {
4
5     static boolean searchKey(NodeSLL_2511531014 head, int key) {
6         NodeSLL_2511531014 curr = head;
7
8         while (curr != null) {
9             if (curr.data_1014 == key)
10                return true;
11            curr = curr.next_1014;
12        }
13        return false;
14    }
15
16    public static void traversal(NodeSLL_2511531014 head) {
17        NodeSLL_2511531014 curr = head;
18
19        while (curr != null) {
20            System.out.print(" " + curr.data_1014);
21            curr = curr.next_1014;
22        }
23        System.out.println();
24    }
25
26    public static void main(String[] args) {
27        NodeSLL_2511531014 head = new NodeSLL_2511531014(14);
28        head.next_1014 = new NodeSLL_2511531014(21);
29        head.next_1014.next_1014 = new NodeSLL_2511531014(13);
30        head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(30);
31        head.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(10);
32
33        System.out.print("Penelusuran SLL : ");
34        traversal(head);
35
36        int key = 30;
37        System.out.print("cari data " + key + " = ");
38    }
39 }

```

```

37         int key = 30;
38         System.out.print("cari data " + key + " = ");
39
40         if (searchKey(head, key))
41             System.out.println("ketemu");
42         else
43             System.out.println("tidak ada");
44     }
45 }

```

• Pembahasan

Program ini dibuat dengan mendefinisikan class PencarianSLL_2511531014 yang berfungsi untuk melakukan pencarian data pada struktur data *Singly Linked List* (SLL). Program ini memanfaatkan class NodeSLL_2511531014 sebagai node yang menyimpan data dan pointer ke node berikutnya.

1. Method searchKey(NodeSLL_2511531014 head, int key) digunakan untuk mencari suatu nilai dalam linked list. Proses pencarian dilakukan dengan menelusuri setiap node menggunakan perulangan while mulai dari head hingga node terakhir. Jika ditemukan data yang sesuai dengan key, method akan mengembalikan nilai true, sedangkan jika tidak ditemukan hingga akhir list, method akan mengembalikan false.

```

static boolean searchKey(NodeSLL_2511531014 head, int key) {
    NodeSLL_2511531014 curr = head;

    while (curr != null) {
        if (curr.data_1014 == key)
            return true;
        curr = curr.next_1014;
    }

    return false;
}

```

2. Method traversal(NodeSLL_2511531014 head) digunakan untuk menampilkan seluruh isi linked list. Proses dilakukan dengan menelusuri node satu per satu menggunakan perulangan while, kemudian setiap data dicetak ke layar hingga mencapai node terakhir.

```

public static void traversal(NodeSLL_2511531014 head) {
    NodeSLL_2511531014 curr = head;

    while (curr != null) {
        System.out.print(" " + curr.data_1014);
        curr = curr.next_1014;
    }
    System.out.println();
}

```

3. Pada method main, dibuat linked list secara manual dengan beberapa node yang saling terhubung. Setelah itu, method traversal() dipanggil untuk menampilkan seluruh data yang ada dalam linked list.

```

public static void main(String[] args) {
    NodeSLL_2511531014 head = new NodeSLL_2511531014(14);
    head.next_1014 = new NodeSLL_2511531014(21);
    head.next_1014.next_1014 = new NodeSLL_2511531014(13);
    head.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(30);
    head.next_1014.next_1014.next_1014.next_1014 = new NodeSLL_2511531014(10);

    System.out.print("Penelusuran SLL : ");
    traversal(head);
}

```

4. Selanjutnya, dilakukan proses pencarian dengan menentukan nilai key, kemudian method `searchKey()` dipanggil untuk mengecek apakah nilai tersebut terdapat dalam linked list. Hasil pencarian ditampilkan menggunakan kondisi if-else untuk menunjukkan apakah data ditemukan atau tidak.

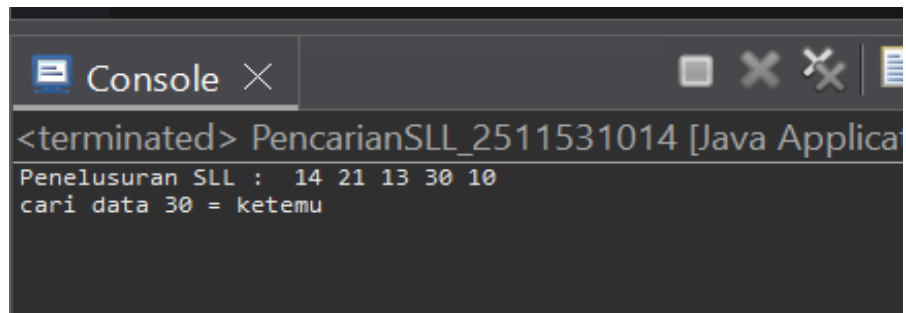
```

int key = 30;
System.out.print("cari data " + key + " = ");

if (searchKey(head, key))
    System.out.println("ketemu");
else
    System.out.println("tidak ada");
}
}

```

5. Secara keseluruhan, program ini mengimplementasikan operasi pencarian pada struktur data Singly Linked List dengan cara penelusuran linear, Hasil Output sebagai berikut



```

<terminated> PencarianSLL_2511531014 [Java Applica
Penelusuran SLL : 14 21 13 30 10
cari data 30 = ketemu

```

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi struktur data *Singly Linked List* (SLL) telah berhasil dilakukan dengan baik serta mampu merepresentasikan konsep Abstract Data Type (ADT) berbasis pointer. Struktur data ini memungkinkan penyimpanan data secara dinamis, di mana setiap elemen (node) saling terhubung melalui referensi ke node berikutnya.

Melalui berbagai program yang telah dibuat, seperti pembuatan node, penambahan data (di awal, akhir, dan posisi tertentu), pencarian data, serta penghapusan node (di awal, akhir, dan posisi tertentu), dapat disimpulkan bahwa operasi pada SLL dilakukan dengan cara menelusuri dan memanipulasi pointer antar node. Hal ini berbeda dengan struktur berbasis array yang menggunakan indeks.

Pada implementasi SLL, kelebihan yang diperoleh adalah fleksibilitas dalam penambahan dan penghapusan data karena tidak memerlukan pergeseran elemen seperti pada array. Namun, proses pencarian dan penelusuran membutuhkan waktu lebih karena harus dilakukan secara berurutan dari node pertama hingga node yang dituju.

Dengan adanya berbagai operasi yang telah diuji, seperti traversal, insertion, deletion, dan searching, dapat disimpulkan bahwa struktur data *Singly Linked List* sangat efektif digunakan dalam pengelolaan data yang bersifat dinamis dan sering mengalami perubahan.

Secara keseluruhan, praktikum ini memberikan pemahaman yang lebih mendalam mengenai konsep linked list, cara kerja pointer antar node, serta penerapannya dalam pemrograman Java, sehingga dapat meningkatkan kemampuan dalam merancang dan mengelola struktur data secara efisien dan terstruktur.

TUGAS PRAKTIKUM PEKAN 5

Mengimplementasikan konsep Single Linked List ke dalam sebuah program dengan studi kasus simulasi antrian pasien rumah sakit.

Topik : Implementasi Single Linked List

Tema : Simulasi Antrian Rumah Sakit

Nama Class “Pasien_2511531014”

- **Kode Program**

```
1 package TugasPekan5_2511531014;
2
3 public class Pasien_2511531014 {
4     String namaPasien_1014;
5     String keluhan_1014;
6     int nomorAntrian_1014;
7     Pasien_2511531014 next_1014;
8
9     // Constructor
10    public Pasien_2511531014(String nama, String keluhan, int nomor) {
11        this.namaPasien_1014 = nama;
12        this.keluhan_1014 = keluhan;
13        this.nomorAntrian_1014 = nomor;
14        this.next_1014 = null;
15    }
16 }
17
```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan class Pasien_2511531014 yang digunakan sebagai representasi node pada struktur data *Singly Linked List* (SLL) dalam program antrian rumah sakit. Class ini berfungsi untuk menyimpan data setiap pasien yang masuk ke dalam antrian.

1. Class ini memiliki atribut namaPasien_1014, keluhan_1014, dan nomorAntrian_1014 yang digunakan untuk menyimpan informasi pasien, yaitu nama pasien, keluhan yang dialami, serta nomor antrian pasien.

```
public class Pasien_2511531014 {
    String namaPasien_1014;
    String keluhan_1014;
    int nomorAntrian_1014;
```

2. Selain atribut data, class ini juga memiliki atribut Pasien_2511531014 next_1014 yang berfungsi sebagai pointer atau referensi ke node pasien berikutnya dalam linked list, sehingga setiap node dapat saling terhubung membentuk antrian.

```

        this.nomorAntrian_1014,
        Pasien_2511531014 next_1014;

```

3. Constructor `Pasien_2511531014(String nama, String keluhan, int nomor)` digunakan untuk menginisialisasi data pasien saat objek dibuat. Nilai parameter akan disimpan ke atribut class menggunakan keyword `this`, sedangkan atribut `next_1014` diatur bernilai `null` sebagai penanda bahwa node belum terhubung dengan node lain.

```

// Constructor
public Pasien_2511531014(String nama, String keluhan, int nomor) {
    this.namaPasien_1014 = nama;
    this.keluhan_1014 = keluhan;
    this.nomorAntrian_1014 = nomor;
    this.next_1014 = null;
}

```

4. Secara keseluruhan, class ini digunakan sebagai node penyusun struktur data Singly Linked List pada program antrian rumah sakit, di mana setiap node menyimpan informasi pasien dan pointer ke pasien berikutnya sehingga data dapat dikelola secara dinamis dan terstruktur. Untuk menjalankan nya kita butuh class Driver.

Nama Class Driver “RumahSakit_2511531014”

- **Kode Program**

```

1 package TugasPekan5_2511531014;
2 import java.util.Scanner;
3 public class RumahSakit_2511531014 {
4     Pasien_2511531014 head_1014;
5     int counter_1014 = 1;
6
7     // Insert (Tambah pasien)
8     public void insert_1014(String nama, String keluhan) {
9         Pasien_2511531014 baru = new Pasien_2511531014(nama, keluhan, counter_1014++);
10
11         if (head_1014 == null) {
12             head_1014 = baru;
13         } else {
14             Pasien_2511531014 temp = head_1014;
15             while (temp.next_1014 != null) {
16                 temp = temp.next_1014;
17             }
18             temp.next_1014 = baru;
19         }
20
21         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + baru.nomorAntrian_1014);
22     }
23
24     // Delete Head (Panggil pasien)
25     public void deleteHead_1014() {
26         if (head_1014 == null) {
27             System.out.println("Antrian kosong!");
28         } else {
29             System.out.println("Memanggil pasien: " + head_1014.namaPasien_1014);
30             head_1014 = head_1014.next_1014;
31         }
32     }
33
34     // Display (Tampilkan Antrian)
35     public void display_1014() {
36         if (head_1014 == null) {
37             System.out.println("Antrian kosong!");
38             return;
39         }
40     }

```

```

41     Pasien_2511531014 temp = head_1014;
42     while (temp != null) {
43         System.out.println("No: " + temp.nomorAntrian_1014 +
44             " | Nama: " + temp.namaPasien_1014 +
45             " | Keluhan: " + temp.keluhan_1014);
46         temp = temp.next_1014;
47     }
48 }
49
50 // Search (Cari Pasien)
51 public void search_1014(String nama) {
52     Pasien_2511531014 temp = head_1014;
53     while (temp != null) {
54         if (temp.namaPasien_1014.equalsIgnoreCase(nama)) {
55             System.out.println("Pasien ditemukan!");
56             System.out.println("No: " + temp.nomorAntrian_1014 +
57                 " | Keluhan: " + temp.keluhan_1014);
58             return;
59         }
60         temp = temp.next_1014;
61     }
62     System.out.println("Pasien tidak ditemukan!");
63 }
64
65 // Status antrian
66 public void status_1014() {
67     if (head_1014 == null) {
68         System.out.println("Antrian kosong!");
69     } else {
70         int jumlah = 0;
71         Pasien_2511531014 temp = head_1014;
72         while (temp != null) {
73             jumlah++;
74             temp = temp.next_1014;
75         }
76         System.out.println("Jumlah pasien: " + jumlah);
77         System.out.println("Pasien terdepan: " + head_1014.namaPasien_1014);
78     }
79 }
80

```

```

81
82 //Kelas Main
83 public static void main(String[] args) {
84     Scanner input = new Scanner(System.in);
85     RumahSakit_2511531014 rs = new RumahSakit_2511531014();
86
87     int pilihan;
88
89     do {
90         System.out.println("\n=== ANTRIAN RUMAH SAKIT ===");
91         System.out.println("1. Daftarkan Pasien");
92         System.out.println("2. Panggil Pasien");
93         System.out.println("3. Tampilkan Antrian");
94         System.out.println("4. Cari Pasien");
95         System.out.println("5. Status Antrian");
96         System.out.println("6. Keluar");
97         System.out.print("Pilihan: ");
98         pilihan = input.nextInt();
99         input.nextLine();
100
101         switch (pilihan) {
102             case 1:
103                 System.out.print("Nama: ");
104                 String nama = input.nextLine();
105                 System.out.print("Keluhan: ");
106                 String keluhan = input.nextLine();
107                 rs.insert_1014(nama, keluhan);
108                 break;
109
110             case 2:
111                 rs.deleteHead_1014();
112                 break;
113
114             case 3:
115                 rs.display_1014();
116                 break;
117

```

```

118             case 4:
119                 System.out.print("Cari nama: ");
120                 String cari = input.nextLine();
121                 rs.search_1014(cari);
122                 break;
123
124             case 5:
125                 rs.status_1014();
126                 break;
127
128             case 6:
129                 System.out.println("Keluar...");
130                 break;
131
132             default:
133                 System.out.println("Pilihan tidak valid!");
134         }
135     } while (pilihan != 6);
136 }
137 }
138

```

- **Pembahasan**

Program ini dibuat dengan mendefinisikan class RumahSakit_2511531014 yang digunakan untuk mengelola antrian pasien menggunakan struktur data *Singly Linked List* (SLL). Pada bagian awal program dilakukan import java.util.Scanner yang digunakan untuk menerima input dari pengguna secara interaktif.

1. Class ini memiliki atribut Pasien_2511531014 head_1014 yang berfungsi sebagai penunjuk node pertama dalam linked list, serta atribut counter_1014 yang digunakan untuk memberikan nomor antrian otomatis kepada setiap pasien yang mendaftar.

```
1 : TugasPekan5_2511531014;  
2 : java.util.Scanner;  
3 : class RumahSakit_2511531014 {  
4 :     Pasien_2511531014 head_1014;  
5 :     int counter_1014 = 1;  
6 }
```

2. Method insert_1014(String nama, String keluhan) digunakan untuk menambahkan pasien ke dalam antrian. Method ini membuat objek pasien baru menggunakan constructor Pasien_2511531014, kemudian jika antrian kosong node baru langsung menjadi head. Jika antrian sudah memiliki data, dilakukan penelusuran hingga node terakhir menggunakan perulangan while, lalu node baru ditambahkan di bagian akhir antrian sesuai prinsip FIFO (*First In First Out*).

```
// Insert (Tambah pasien)  
public void insert_1014(String nama, String keluhan) {  
    Pasien_2511531014 baru = new Pasien_2511531014(nama, keluhan, counter_1014++);  
  
    if (head_1014 == null) {  
        head_1014 = baru;  
    } else {  
        Pasien_2511531014 temp = head_1014;  
        while (temp.next_1014 != null) {  
            temp = temp.next_1014;  
        }  
        temp.next_1014 = baru;  
    }  
  
    System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + baru.nomorAntrian_1014);  
}
```

3. Method deleteHead_1014() digunakan untuk memanggil sekaligus menghapus pasien yang berada di posisi paling depan. Jika antrian kosong maka program menampilkan pesan bahwa antrian kosong, sedangkan jika terdapat data maka pasien terdepan dipanggil dan head dipindahkan ke node berikutnya.

```

// Delete Head (Panggil Pasien)
public void deleteHead_1014() {
    if (head_1014 == null) {
        System.out.println("Antrian kosong!");
    } else {
        System.out.println("Memanggil pasien: " + head_1014.namaPasien_1014);
        head_1014 = head_1014.next_1014;
    }
}

```

4. Method `display_1014()` digunakan untuk menampilkan seluruh data pasien yang berada dalam antrian. Method ini melakukan traversal mulai dari head hingga node terakhir dan menampilkan nomor antrian, nama pasien, serta keluhan pasien secara berurutan.

```

// Display (Tampilkan Antrian)
public void display_1014() {
    if (head_1014 == null) {
        System.out.println("Antrian kosong!");
        return;
    }

    Pasien_2511531014 temp = head_1014;
    while (temp != null) {
        System.out.println("No: " + temp.nomorAntrian_1014 +
            " | Nama: " + temp.namaPasien_1014 +
            " | Keluhan: " + temp.keluhan_1014);
        temp = temp.next_1014;
    }
}

```

5. Method `search_1014(String nama)` digunakan untuk mencari pasien berdasarkan nama. Penelusuran dilakukan secara linear menggunakan perulangan `while`. Jika nama pasien ditemukan, program menampilkan informasi pasien tersebut, sedangkan jika tidak ditemukan maka program menampilkan pesan bahwa pasien tidak ditemukan.

```

// Search (Cari Pasien)
public void search_1014(String nama) {
    Pasien_2511531014 temp = head_1014;
    while (temp != null) {
        if (temp.namaPasien_1014.equalsIgnoreCase(nama)) {
            System.out.println("Pasien ditemukan!");
            System.out.println("No: " + temp.nomorAntrian_1014 +
                " | Keluhan: " + temp.keluhan_1014);
            return;
        }
        temp = temp.next_1014;
    }
    System.out.println("Pasien tidak ditemukan!");
}

```

6. Method `status_1014()` digunakan untuk menampilkan kondisi antrian, yaitu jumlah pasien yang sedang mengantri dan pasien yang berada di posisi paling depan. Jumlah pasien dihitung dengan melakukan traversal terhadap seluruh node dalam linked list.

```

// Status antrian
public void status_1014() {
    if (head_1014 == null) {
        System.out.println("Antrian kosong!");
    } else {
        int jumlah = 0;
        Pasien_2511531014 temp = head_1014;
        while (temp != null) {
            jumlah++;
            temp = temp.next_1014;
        }
        System.out.println("Jumlah pasien: " + jumlah);
        System.out.println("Pasien terdepan: " + head_1014.namaPasien_1014);
    }
}
}

```

7. Pada method main, dibuat objek Scanner untuk menerima input pengguna dan objek RumahSakit_2511531014 sebagai antrian utama rumah sakit. Program menggunakan perulangan do-while untuk menampilkan menu secara terus menerus hingga pengguna memilih keluar.

```

//Kelas Main
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    RumahSakit_2511531014 rs = new RumahSakit_2511531014();

    int pilihan;

    do {
        System.out.println("\n=== ANTRIAN RUMAH SAKIT ===");
        System.out.println("1. Daftarkan Pasien");
        System.out.println("2. Panggil Pasien");
        System.out.println("3. Tampilkan Antrian");
        System.out.println("4. Cari Pasien");
        System.out.println("5. Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan = input.nextInt();
        input.nextLine();
    }
}

```

8. Struktur switch-case digunakan untuk menjalankan menu sesuai pilihan pengguna, yaitu menambah pasien, memanggil pasien, menampilkan antrian, mencari pasien, melihat status antrian, dan keluar dari program. Setiap pilihan akan memanggil method yang sesuai sehingga program dapat berjalan secara interaktif.

```

90         switch (pilihan) {
91             case 1:
92                 System.out.print("Nama: ");
93                 String nama = input.nextLine();
94                 System.out.print("Keluhan: ");
95                 String keluhan = input.nextLine();
96                 rs.insert_1014(nama, keluhan);
97                 break;
98             case 2:
99                 rs.deleteHead_1014();
100                break;
101             case 3:
102                 rs.display_1014();
103                 break;
104             case 4:
105                 System.out.print("Cari nama: ");
106                 String cari = input.nextLine();
107                 rs.search_1014(cari);
108                 break;
109             case 5:
110                 rs.status_1014();
111                 break;
112             case 6:
113                 System.out.println("Keluar...");
114                 break;
115             default:
116                 System.out.println("Pilihan tidak valid!");
117         }
118     } while (pilihan != 6);
119 }

```

9. Secara keseluruhan, program ini mengimplementasikan Abstract Data Type (ADT) Queue menggunakan struktur data Singly Linked List dalam sistem antrian rumah sakit, sehingga pengelolaan data pasien dapat dilakukan secara dinamis, terstruktur, dan sesuai prinsip FIFO, Berikut Adalah hasil Outputnya.

```

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 1
Nama: nay
Keluhan: flu
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 2
Memanggil pasien: nay

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 3
Antrian kosong!

```

```
=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 4
Cari nama: nay
Pasien tidak ditemukan!

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 5
Antrian kosong!

=== ANTRIAN RUMAH SAKIT ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Status Antrian
6. Keluar
Pilihan: 6
Keluar...
```